

Retrieval-Ready Study Assistant

for NCERT Science

WEEK 9 MINI-PROJECT

PG Diploma in AI-ML & Agentic AI Engineering · Cohort 2

This document is for you, the student. It contains everything you need to design, build, and evaluate this week's mini-project — plus hints from senior engineers that you will not find in tutorials, and a reflection questionnaire that helps you see honestly where you stand.

A NOTE BEFORE YOU BEGIN

You are at different starting points. Some of you will finish Base comfortably; some will reach Advanced; some will struggle through Base and learn the most of anyone. That is fine.

This project is not a competition. It is a chance to **apply** what you have been studying, to **feel** which parts of it you really understand, and to **practice** talking about your work the way you will need to in interviews. The reflection questionnaire at the end is a tool for you first, and a grading signal for us second. Answer it honestly.

Effort and honesty matter more than polish. A student who completes Base thoughtfully, reports real failures, and writes an honest reflection will learn more — and score higher — than a student who rushes through Advanced with a shallow evaluation.

What is in this document

- **Section 1.** The scenario (PariShiksha)
- **Section 2.** What concepts this project exercises
- **Section 3.** Environment, data, and setup
- **Section 4.** Task ladder (Base / Stretch / Advanced / Open)
- **Section 5.** Expert hints (non-obvious things that trip most people up)
- **Section 6.** What "done" looks like at each tier
- **Section 7.** Submission and rubric
- **Section 8.** Reflection questionnaire (submit with the project)
- **Section 9.** Interview-flavor scenarios to practice with
- **Section 10.** One closing thought

1. The scenario

PariShiksha is a 4-year-old edtech startup operating across Tier-2 and Tier-3 cities in Rajasthan, Madhya Pradesh, and Uttar Pradesh. They run after-school NCERT-aligned study centres for Class 9 and 10 students, with about 4,200 students currently enrolled across 18 centres. Parents pay a modest monthly fee. Margins are thin. Tutor availability is the bottleneck on growth.

The founders want to pilot a bounded study assistant: something that answers NCERT Science questions reliably when a tutor is not available. The non-negotiable is that the assistant must stay grounded in NCERT content. A parent whose daughter was told something contradicting the textbook would escalate, and the trust damage would kill the pilot.

You are the sole ML engineer on a three-person prototype team. A senior backend engineer handles infrastructure. A Class 9 Science teacher is on contract to help build the evaluation set. You have this week to build the retrieval-ready foundation — a system that extracts, structures, and queries one or two chapters of NCERT Science content, generates grounded answers, and can be honestly evaluated on whether it works.

You will **not** build a full vector-database RAG pipeline this week. That is next week's project. This week is about making the content retrieval-ready, the generation grounded, and the evaluation real. If this foundation is weak, next week's work will collapse onto it.

WHY THE SCENARIO MATTERS

Every week's project is built around a realistic context, not a generic exercise. The scenario shapes what "correct" means for your system. PariShiksha serves students in cities where Hindi-English code-switching is normal, where data connectivity is unreliable, and where tutor trust is the product's moat. Keep this in mind when you design your evaluation and when you write your reflection.

2. What this project exercises

This week you have been studying tokenization, encoder-decoder architectures, transformer internals, generative models (including GANs as a contrast), and the major transformer variants. The project is designed so that each of those topics shows up as a practical decision you must make — not as a section in a report.

Topic	Where it appears in the project
Tokenization	Your chunking strategy begins with a tokenizer choice. Different tokenizers split scientific terms (photosynthesis, specific-heat-capacity) differently. Stage 1 asks you to compare at least two.
Encoder-decoder models	Stretch asks you to try a T5-family (encoder-decoder) model alongside a decoder-LLM. You will feel the difference in generation behavior on the same retrieved context.
Attention and context	Chunks that are too large waste context window and dilute attention; chunks that are too small lose surrounding reasoning. Your chunk-size experiment in Stretch surfaces this directly.
Transformer variants	Three architectural families sit behind three choices: BERT-style extractive QA, T5-style generative QA, and decoder-only LLMs (like Gemini). The project lets you compare at least two of them on the same evaluation set.
GANs (this week's odd one out)	GANs are not used in this project. Reflection prompt 3 asks you to explain why GAN-style adversarial generation is the wrong tool for grounded textbook QA. The answer teaches you something important about matching generative architectures to problems.
Evaluation discipline	Stage 4 is where most students discover how little they know about their own system. Your evaluation set is the measure of your engineering maturity this week.

And what it sets up for next week

Next week is advanced RAG — dense embeddings, hybrid retrieval, cross-encoder reranking, guardrails, and production concerns. All of that composes on top of this week's foundation. Students who build this week carelessly will spend next week fighting bugs they introduced now. Students who build this week well start next week with design judgment.

3. Environment, data, and setup

3.1 Hardware

- Disk: ~2 GB free for cached models and corpus data
- RAM: 8 GB minimum; 16 GB comfortable
- Compute: CPU is sufficient. You do not need a GPU for this project.
- Python 3.10 or newer, in a fresh virtual environment

3.2 Required libraries

- **pymupdf** or **pdfplumber** — PDF extraction
- **transformers** and **tokenizers** — Hugging Face
- **torch** — backend; CPU is fine
- **rank_bm25** — lexical retrieval (alternative: **scikit-learn** TF-IDF)
- One LLM API client: **google-generativeai** (Gemini free tier, recommended), **openai**, or **anthropic**
- Optional: **ollama** if you want to run models locally (not required)

3.3 LLM backend

Default: use an LLM API for generation. Gemini's free tier handles this task comfortably and needs no infrastructure. API-key setup instructions will be circulated separately.

The focus this week is retrieval and grounding, not model serving. Using an API keeps your hands on the interesting problem. If you want to try Ollama locally, that is fine, but it will cost you time you would otherwise spend on evaluation.

3.4 Corpus

Pick one or two chapters from the NCERT Class 9 Science textbook. Good starting points: **Motion, Force and Laws of Motion**, or **Gravitation**. These contain a mix of conceptual explanations, worked examples, and end-of-chapter questions, which is what you want for testing different retrieval behaviors.

Primary source:

<https://ncert.nic.in/textbook.php?iesc1=0-11>

Chapter files are named `iesc1XX.pdf` where `XX` is the chapter number. Link to the source in your README. Do not commit the PDF files to your repo.

Backup (only if you cannot reach ncert.nic.in):

OpenStax College Physics, CC-BY licensed: github.com/philschatz/physics-book

EXPERT HINT · Why this content is appropriately messy

You will not find this corpus pre-cleaned on Kaggle or Hugging Face. The messiness is the point, and it comes from PDF extraction itself:

- Formula rendering is inconsistent — some equations come through as text, some as images, some as mojibake
- Figure captions reference diagrams you will not have, creating dangling references
- Section boundaries are not cleanly marked in extracted text
- Concept paragraphs, worked examples, and end-of-chapter questions have different structures, but flat extraction flattens them into one stream

A submission that says "the corpus was clean" is a submission that did not look at it carefully.

3.5 Collaboration policy

Collaborating with teammates on environment setup is fine and encouraged — it is not the part being graded. Using AI tools to debug setup issues is fine. The experiments, the evaluation, and the reflection are your own work.

4. Task ladder

You progress from Base outward. Base is required; Stretch, Advanced, and Open are extensions. Nothing is gated — you attempt higher tiers when you are ready, not when someone unlocks them.

Base (required)

Five stages, executed in order. Each stage produces evidence in your notebook.

Stage 1 — Corpus

- Extract text from one or two chapters using PyMuPDF or pdfplumber
- Split the text into at least three content types where possible: concept paragraphs, worked examples, end-of-chapter questions
- Compare at least two tokenizers (GPT-2 BPE, BERT WordPiece, T5 SentencePiece) on five representative passages — report token counts and where boundaries disagree
- Design a chunking strategy (size, overlap, handling) and justify the choices in 150–250 words

Stage 2 — Retrieval

- Build a chunk store where each chunk carries metadata: chapter name, section, content type
- Implement a lexical retriever using BM25 (preferred) or TF-IDF, returning top-k chunks for a query
- Test with three questions from your chapters and confirm the retrieved chunks are relevant

Stage 3 — Generation

- Integrate an LLM (API recommended) for answer generation
- Design a grounding prompt that instructs the LLM to answer only from provided context and to refuse when the answer is not in context
- Produce a callable function `answer(question) -> {answer, retrieved_chunks}`

Stage 4 — Evaluation

- Build an evaluation set of 15–20 questions: 10–12 direct from textbook, 2–3 paraphrased, 3–5 out-of-scope
- Score each question on three axes: correctness (yes/no/partial), grounding (is the answer supported by retrieved chunks?), refusal-appropriateness (for out-of-scope only)
- Save results as `evaluation_results.md` or `.csv`
- Identify three working examples and two failing examples; for each failure, write one sentence on probable cause

Stretch

- **Model family comparison.** Run the same evaluation set through at least two model families: Extractive QA (deepset/roberta-base-squad2), encoder-decoder (google/flan-t5-small or flan-t5-base), decoder-LLM (the API you chose). Pick any two. Record differences in correctness and grounding.
- **Attention inspection.** Pick one question where flan-t5-small answered correctly and one where it got it wrong. Visualize attention on the encoder's final layer. Write 100–150 words on what you saw.

- **Chunking experiment.** Test at least two chunk sizes (e.g., 250 vs 500 tokens) and report impact on evaluation scores.

Advanced

- **Dense retrieval bridge.** Add an embedding-based retriever using sentence-transformers/all-MiniLM-L6-v2 or similar. Compare against BM25.
- **Guardrails.** Implement explicit handling for out-of-scope questions, prompt-injection attempts, and malformed inputs.
- **Failure-mode document.** Produce a 400–600 word `failure_modes.md` covering the top three likely production failure modes, grounded in actual eval results, not speculation.

Open (bonus)

Pick one. Must be feasible in ~4–6 additional hours.

- **Teacher mode:** answers cite specific chunks with page/section references
- **Paraphrase robustness:** generate 20 paraphrased versions of textbook questions and measure retrieval stability
- **Multi-chapter query:** handle questions that span two chapters (e.g., "how do force and work relate?")
- **Multimodal stretch:** identify one figure caption in the PDF and handle a question that references it

5. Expert hints

These are the kinds of things a senior engineer would tell you across a table if you said "I'm building a RAG system for the first time." None of them are in the standard tutorials. Each one matches a mistake most people make on their first attempt. Read them once now and again after you have a working Base.

EXPERT HINT · Retrieval-time and index-time tokenization must match

If you tokenize your corpus with one tokenizer (BPE) but tokenize your queries with another (WordPiece), your BM25 scores are meaningless. For BM25 this rarely bites because you probably use a simple word-tokenizer both sides. But the moment you add a neural reranker or dense retriever next week, this asymmetry will silently corrupt your scores. Make tokenizer choice an explicit decision, not an accident of which library you called.

EXPERT HINT · Chunk boundaries that split an example from its solution destroy retrieval

If chunk 7 contains "Example: a car moving at 20 m/s..." and chunk 8 contains the worked solution, a student asking "how do I solve the car example?" retrieves chunk 7 (the problem) but not chunk 8 (the answer). The LLM then confidently generates a wrong solution. Before you set your chunk size, look at your actual content and identify which semantic units must stay together. Overlap is your friend; pure fixed-size chunking is a blunt tool.

EXPERT HINT · The most common hallucination is not "model invents facts"

It is "model dutifully generates from whatever context it was given, even if that context is irrelevant." When your system hallucinates, your first instinct will be to blame the LLM. Nine times out of ten the bug is upstream — your retriever returned a plausible-looking but wrong chunk, and the LLM did exactly what you told it to. Always print the retrieved chunks for a failing query before you touch the prompt or change the model.

EXPERT HINT · "Only answer from context" is weaker than "refuse if not in context"

The first instruction is permissive — the LLM can interpret it as "prefer the context." The second is a constraint — the LLM must check whether the answer is actually present, and refuse if it isn't. Most grounding failures come from the first phrasing. Test both explicitly. You will see a difference.

EXPERT HINT · Set temperature = 0 for evaluation, or your scores are not reproducible

If you score your system at 14/20 today and 11/20 tomorrow with the same eval set and no code changes, the problem is decoding non-determinism. For evaluation use temperature=0 (or the API's equivalent). For the user-facing experience you might want some temperature; those are two different things. Never confuse them.

EXPERT HINT · The gap between good chunking and bad chunking is larger than the gap between LLMs

If your Base scores 12/20 with Gemini and GPT-4 would have scored 14/20 on the same questions, that is a 10% improvement. If better chunking would have taken the same Gemini system to 17/20, that is a 25% improvement. This week, spend your time on your chunks and your prompt. Do not chase LLM quality before fixing the upstream.

EXPERT HINT · flan-t5-small will hallucinate on reasoning questions because of its size

flan-t5-small has 77 million parameters. For comparison, Gemini 1.5 Flash has on the order of billions. If you use flan-t5-small on a question that requires multi-step reasoning over the context ("which of these forces is greater and why?"), it will often produce fluent but wrong answers. This is not a prompt problem; it is a capacity problem. Use flan-t5-small for comparison, for structural analysis, for showing your variant-comparison — not as the production choice for this use case.

EXPERT HINT · When iterating on your grounding prompt, change one variable at a time

The strongest temptation when evaluation is going badly is to rewrite the prompt, change the chunk size, and swap the model all in the same commit. Then the next eval run is better and you do not know which change helped. Engineers who have shipped real systems know that single-variable iteration is slower in the short term and faster in the medium term. Commit after each change. Your git log becomes your experiment log.

EXPERT HINT · Out-of-scope questions that retrieve relevant-looking chunks are the real adversarial test

The easy out-of-scope test is "Who is the Prime Minister of India?" — your retriever returns nothing relevant, the LLM refuses, you claim victory. The real test is "Explain quantum entanglement in Chapter 9 of NCERT" — your retriever returns Chapter 9 content that is not about quantum entanglement, and a weak grounding prompt will happily synthesize a confident wrong answer from unrelated context. Include at least one of these in your eval set. This is the question that separates systems that pilot from systems that don't.

EXPERT HINT · Real student questions are messier than the ones you will write yourself

If you build your own eval set, you will unconsciously phrase questions the way a textbook would. Real students type questions with grammar errors, missing punctuation, Hindi-English code-switching, and half-remembered terms ("what is newton 2nd law force equal mass time acceleration"). If time permits, ask someone outside the cohort to write five questions for your eval set without looking at the textbook. The quality drop will teach you something.

6. What "done" looks like

Base complete

A runnable notebook that extracts one chapter, compares two tokenizers, builds a BM25-retrieved chunk store, generates grounded answers via an LLM API, and produces a real evaluation table on 15–20 questions with at least 10 correct, 3+ refusals handled appropriately, and 2 honest failure cases analyzed.

Stretch complete

Base plus a comparison across two model families with measurable score differences, plus one attention inspection, plus one chunking experiment — with a written conclusion about which configuration you would take forward.

Advanced complete

Stretch plus a dense-retrieval comparison, explicit guardrails, and a failure-mode document that reads as if written for a real product team.

NOTE ON EFFORT VS. TIER

A thoughtful Base submission will often score higher than a rushed Advanced submission. The rubric weights evaluation quality and grounding because those are what decide whether this pilot survives. Do the work in order; do not skip to the glossy parts.

7. Submission and rubric

7.1 What to submit

Submit a public GitHub repository link via LMS by **Sunday 9 PM IST**. Repository must contain:

Required files:

- `README.md` — setup and run instructions, link to NCERT source (not committed)
- `notebook.ipynb` — runs end-to-end on a fresh clone
- `evaluation_results.md` or `.csv` — your 15–20 question evaluation table
- `reflection.md` — the reflection questionnaire, completed (Section 8 of this document)

Optional files (only if attempted):

- `failure_modes.md` for Advanced
- One small supporting artifact for Open

SUBMISSION HYGIENE GATE (PASS/FAIL)

Required files present. Notebook runs on a fresh clone. At least 3 intermediate commits in git history. Missing any of these zeroes the project regardless of experiment quality. Reproducibility is a real skill we are testing.

7.2 Rubric summary

#	Criterion	Weight
1	Problem framing and corpus structuring	15
2	Tokenization and chunking logic	20
3	Retrieval and grounded answering	20
4	Evaluation quality and failure analysis	20
5	Architecture comparison and model reasoning	15
6	Reflection and tradeoff discussion	10
	Total	100

Advanced components (dense retrieval, guardrails, failure-modes document) earn up to 5 bonus points. Open extension earns up to 5 bonus points. Bonus can push above 100.

8. Reflection questionnaire

Submit this, completed, as `reflection.md` in your repo. Target length: 700–1000 words total across all answers.

EXPERT HINT · About this questionnaire

The questions below are designed to be answered by you, from your specific implementation. They ask about things only you know — what you tried, what broke, what you observed, what you decided, and why. An LLM can write fluent text about RAG; it cannot tell the grader what happened in *your* notebook on Wednesday afternoon.

Answer honestly. A short, specific, honest answer is worth more than a long, vague, polished one. If you did not do something, say so — that is more useful to you and to us than pretending otherwise. This questionnaire is meant to help you reflect, practice interview-style articulation, and give us a signal on where you stand. It is not a trap.

Part A — Your implementation artifacts

These questions reference your actual code, your actual eval output, and your actual git history.

A1. Your chunking parameters

Paste the exact chunking parameters you settled on (chunk size, overlap, any special handling for content types).

Then: what was the one experiment or observation that pushed you to those specific values? Be specific — do not say "I tried a few options." Name the options and say what you observed.

A2. A retrieved chunk that was wrong for its query

Copy and paste one retrieved chunk from your system that was the wrong chunk for the query that retrieved it. Then explain, in two to three sentences, why your retriever returned it. The answer is almost never "the retriever is bad" — trace the specific cause.

A3. Your grounding prompt, v1 and v(final)

Paste the first version of your grounding prompt and the final version. Between them, describe the single observation (on which specific query, or which specific type of failure) that caused each revision. If you only wrote one version of your prompt, write that here and explain why you didn't need to iterate.

Part B — Numbers from your evaluation

These questions ask about specific numbers you measured. They cannot be guessed.

B1. Your evaluation scores

Out of N questions in your evaluation set, how many were: (a) correct, (b) grounded (answer supported by retrieved chunks), (c) appropriate refusals (for out-of-scope only)? Then: which of these three numbers bothered you most, and why?

B2. If you ran the chunk-size experiment (Stretch)

What were the two chunk sizes you compared? What was the delta on correctness? What was the delta on refusal-appropriateness? If you did not run this experiment, say so and skip to B3.

B3. If you compared model families (Stretch)

Which two did you compare? On which type of question did each family do better? Name one specific question where the two families gave meaningfully different answers, and paste both answers (short — a sentence each is fine).

Part C — Debugging moments

These questions ask about specific moments — they are time-stamped and personal.

C1. The most frustrating bug

Describe the single most frustrating bug you hit this week. Approximately how long did it take to fix? What did you try first (that didn't work)? What was the actual fix? If someone else hit the same bug next week, what would be the fastest way for them to find the fix?

C2. What still bothers you

Even if your system is "mostly working" as you submit, name one specific behavior that still bothers you. Why does it bother you? What would you need to do to fix it?

Part D — Architecture and reasoning

These questions are interview-flavor but grounded in your project. Answer them in your own voice, using specific evidence from your work.

D1. Why not just ChatGPT?

Your hiring manager says: "I can just use ChatGPT for this. Why do I need a retrieval system?" Respond in your own words, using at least one specific example from your evaluation (not a hypothetical one).

D2. The GANs reflection

This week you studied GANs. You did not use GANs for this project. Explain, in your own words, why GAN-style adversarial generation is the wrong tool for this bounded textbook-assistant problem. What is the deeper principle — about matching architectures to problems — that this teaches you?

D3. Honest pilot readiness

PariShiksha's CTO asks you: "Can we launch this pilot with 100 students next Monday?" Based on what your system actually did in evaluation, what is your honest answer? What are the three specific things you would want to verify or fix first?

Part E — Effort and self-assessment

Be honest. There is no wrong answer here — only honest ones and unhelpful ones.

E1. Effort rating

On a scale of 1–10, how would you rate your effort on this project? If you rated it below 7, what got in the way? If you rated it 7 or above, name one specific thing you did that you are genuinely proud of.

E2. The gap between you and a stronger student

Imagine a stronger student in the cohort working on the same project. Name one specific thing they might have done that you did not. Is there a reason you did not? (Time, confidence, something else?)

E3. What would change with two more days

If you had exactly two more days on this project, what is the first thing you would do, and what is the last thing you would do? The order matters — it tells us what you think is most important.

9. Interview-flavor scenarios

These are the kinds of questions you will face in interviews for GenAI Engineer, LLM Application Engineer, and AI Platform roles. They are case-study style — there is no single correct answer, and the interviewer is evaluating how you think, not what you memorized.

You do not need to submit answers to these. Practice them — with a teammate, in front of a mirror, out loud on a walk. The goal is to get comfortable talking through ambiguous technical decisions.

EXPERT HINT · How to approach case-study questions without anxiety

Case-study interviewers are not trying to catch you out. They want to see how you reason under uncertainty. A junior engineer who says "I would start by asking what the current failure modes are, then pick the fix that addresses the most common one" shows better judgment than one who confidently picks an answer without acknowledging the tradeoffs.

Three things that land well: (1) name the thing you would need to know before deciding, (2) walk through your reasoning aloud even when uncertain, (3) acknowledge what you don't know rather than bluff.

Interviewers reward honest thinking over confident guessing. If you have actually built something this week, you have real experiences to draw from — use them.

INTERVIEW FLAVOR · The domain-shift question

"A pharmaceutical company wants a similar RAG system reading FDA regulatory filings. What are three changes you would make compared to your PariShiksha build? Which of those changes is the most important, and why?"

What the interviewer is looking for: can you reason from context to architecture? Good answers identify that pharma means higher liability, cross-document reasoning, exact citation requirements, and possibly multi-lingual (English + EU languages). Great answers name which single change matters most and defend the ranking.

INTERVIEW FLAVOR · The scale question

"Your PariShiksha system works at prototype scale. Walk me through what breaks first as we move from 18 centres to 500 centres and from 15 eval questions to 50,000 queries per day."

What the interviewer is looking for: production thinking. Answers should touch on latency under concurrent load, cost per query at scale (LLM API bills become real), evaluation drift as real queries diverge from your hand-picked set, and monitoring — how would you even know if the system started hallucinating more next month?

INTERVIEW FLAVOR · The tradeoff question

"You have 10 additional hours of engineering time. You must choose exactly one: (a) better chunking, (b) better grounding prompt, (c) dense retrieval, (d) explicit guardrails. Which, and why?"

What the interviewer is looking for: decision-making under constraints. The "correct" answer depends on what your system's current failure modes actually are. The strongest answers say "it depends on what my evaluation shows is currently weakest — if most failures are hallucinations, I pick (b); if most failures are retrieval misses, I pick (a) or (c); if most failures are out-of-scope issues, I pick (d)." Weak answers pick a tier without justification.

INTERVIEW FLAVOR · The failure-mode question

"Tell me about a time a system you built failed in a surprising way. Walk me through how you figured out what was wrong."

What the interviewer is looking for: engineering maturity. This is the question where your reflection questionnaire answers become gold. Students who genuinely debugged their system have a story; students who used an LLM to answer the questionnaire do not. The honest specific story wins every time over the impressive-sounding generic one.

INTERVIEW FLAVOR · The architecture whiteboard question

"Draw the architecture of your PariShiksha system on the whiteboard. Now, if we needed to handle questions in both English and Hindi (because students code-switch), what changes? Draw those changes."

What the interviewer is looking for: can you extend an existing design to a new requirement? Good answers identify that tokenizer behavior on Devanagari differs from English, that embedding models trained primarily on English may degrade, and that evaluation sets must include Hindi queries. Great answers also note that pretrained models vary in multilingual quality and name specific ones (e.g., multilingual MPNet, XLM-R) that would help.

INTERVIEW FLAVOR · The "what if it breaks in production" question

"Your assistant has been live for two weeks. A parent calls, furious, saying the assistant told her daughter that Newton's second law is $F = mv$ instead of $F = ma$. How do you investigate?"

What the interviewer is looking for: ops thinking and debugging discipline. You should be able to sketch a debugging playbook: reproduce the specific query, inspect what chunks were retrieved, check the generation trace, determine whether this is a retrieval miss or a generation error, and propose both immediate mitigation (e.g., add this case to eval set and block deployment until it passes) and root cause fix.

10. One closing thought

The scenario (PariShiksha, Tier-2/3 rollout) is not decoration. The rubric weights evaluation quality and grounding because *those* are what decide whether a pilot like this survives contact with a real parent who finds a hallucination.

A student who builds a working pipeline and then tests it seriously — including questions it *should* refuse — will learn more than a student who builds something fancier and tests it on three easy questions.

Take the evaluation seriously. Write the grounding prompt carefully. Let your system fail on something and trace the cause. That is the work.

— *End of project document* —